

ADMINPRO - RESUMEN TECNICO DEL PROYECTO

RecuperaT Gestion Clinica | 26 de agosto de 2026

1. RESUMEN EJECUTIVO

AdminPro es una plataforma administrativa para una clinica de rehabilitacion. Centraliza pacientes, tratamientos, agenda, caja, inventario, autorizaciones y roles de usuario.

Estado actual: prototipo de interfaz avanzado, con datos simulados en memoria y una base backend preparada parcialmente. Todavia no existen API REST, persistencia de operaciones ni autenticacion real conectada al frontend.

2. ARQUITECTURA GENERAL

- Frontend: React 19, TypeScript, Vite, Tailwind CSS 4, Lucide React.
- Backend: Java 21, Spring Boot, Spring MVC, JPA, Security y Validation.
- Base de datos: PostgreSQL 15 ejecutado mediante Docker Compose.
- Persistencia: entidades JPA definidas, sin repositorios ni servicios.

El frontend funciona como una SPA. La navegacion se controla mediante el estado `currentScreen` en `App.tsx`; no se usa React Router. Los datos iniciales provienen de `frontend/src/data/mockData.ts` y las operaciones actualizan arrays locales mediante React state.

3. INFRAESTRUCTURA

Docker Compose crea PostgreSQL 15, base `adminpro_db`, usuario `adminpro`, puerto 5432 y volumen persistente `pgdata`. Solo se levanta la base de datos; backend y frontend no estan containerizados.

El backend usa `application.yml` para conectar a `localhost:5432`, con `Hibernate ddl-auto=update` y SQL visible en consola. El frontend tiene scripts `dev`, `build`, `preview` y `lint` en `package.json`.

4. BACKEND: MODELO DE DATOS

Paciente: UUID, folio, nombre, telefono, correo, direccion, contacto de emergencia, sucursal de origen, fecha y tipo de ingreso, aseguradora. Relacion ManyToOne con Sucursal.

Usuario: UUID, nombre, credencial unica, rol, sucursales asignadas, estado activo y autenticacion de dos factores. Tiene relacion ManyToOne con Rol y ManyToMany con Sucursal mediante usuario_sucursal.

Rol: UUID y nombre unico. La matriz de permisos esta prevista, pero no existe todavia como entidad o tabla independiente.

Sucursal: UUID, nombre, direccion, telefono y WhatsApp Business.

Las entidades estan en backend/src/main/java/com/recuperat/adminpro/model.
La clase de arranque es AdminProApplication.java.

5. FRONTEND: VISTAS PRINCIPALES

- Login: formulario, login demo, validacion basica y recuperacion simulada.
- Dashboard: indicadores, autorizaciones recientes y pacientes por renovar.
- Pacientes: busqueda, filtros, detalle, alta, agenda y renovacion.
- Agenda: citas con terapeuta, tratamiento, fecha, hora y estado.
- Finanzas: ingresos, ticket promedio, transacciones y cobro en caja.
- Inventario: articulos, stock, minimo, unidad, categoria y alertas.
- Autorizaciones: descuentos, anulaciones, aprobaciones y rechazos.
- Roles: creacion visual de perfiles y seleccion de permisos.

App.tsx coordina sesion, navegacion, busqueda, menu movil, datos, modales y notificaciones toast.

6. COMPONENTES Y DATOS

Componentes de layout: Sidebar, TopBar y AdminProLogo.

Modales: NewPatientModal, PatientDetailModal, NewAppointmentModal y CheckoutModal.

Los tipos TypeScript estan en frontend/src/types.ts e incluyen User, Patient, AuthorizationRequest, Role, Appointment, InventoryItem y Transaction.

mockData.ts contiene el usuario Dr. Silva, pacientes, autorizaciones, roles, citas, inventario y transacciones de demostracion.

7. LIMITACIONES ACTUALES

- El login no consulta al backend: acepta campos no vacios.
- Los cambios se guardan solo en memoria y se pierden al recargar.
- No hay control efectivo de permisos por rol en el servidor.
- Los botones de exportacion y recuperacion usan alert o simulaciones.
- Spring Security esta declarado, pero no tiene configuracion visible.
- Solo existe la prueba contextLoads() en el backend.

8. OBSERVACION DE ESTRUCTURA

App.tsx importa ./components/... mientras el arbol visible ubica los componentes en src/assets/components/.... Debe verificarse esta ruta despues de instalar dependencias y ejecutar el build.

9. PROXIMOS PASOS

Instalar dependencias, confirmar build, crear repositorios, servicios, controladores REST y DTOs; implementar autenticacion y permisos; conectar el frontend a la API; agregar migraciones, pruebas y Docker para todos los servicios.